



Project Planning & Prototyping

J. F. N. Salik,

247-519-VA

**Department of Computer
Engineering Technology**
Vanier College
821 Sainte-Croix Avenue
Saint-Laurent, Québec H4L 3X9

This document was last modified on
Friday 29th May, 2026 at 4:09pm.

<i>Title</i>	Project Planning & Prototyping
<i>Date</i>	May 29, 2026
<i>Author</i>	J. F. N. Salik, (salikj@vaniercollege.qc.ca)
<i>Identification</i>	247-519-VA
<i>Template</i>	Sparkle Version 1.1 for L ^A T _E X 2 _ε



©2026. All rights reserved by J. F. N. Salik,.

This document may not be cited, reproduced, or distributed without explicit
written permission from the author.

Contents

1	Overview: Prototypes, Products & the Business of Engineering	1
1.1	What You Will Build	1
1.2	The Course Map: Two Tracks Over Twelve Weeks	2
1.3	From Prototype to Product: The Gap	3
1.4	Working with Engineers: What You Own	4
1.5	Engineering Businesses by Size	5
1.6	Science for Profit: Cost, Time, and Volume	6
1.7	How a Prototype Becomes Revenue	6
	Review Questions	10
2	The Prototyping Mindset & the Workflow	13
2.1	Prototype = Answer to a Question, Reducer of Risk	14
2.2	The Seven Stages	15
2.2.1	Stage 1: Specification & Risk	15
2.2.2	Stage 2: Scoping & Planning	16
2.2.3	Stage 3: Architecture	16
2.2.4	Stage 4: Subsystem Design	17
2.2.5	Stage 5: Build & Integration	17
2.2.6	Stage 6: Validate & Measure	17
2.2.7	Stage 7: Compile & Economics	18
2.3	The Controlled Iteration Loop (Design ↔ Validate)	18
2.4	Prototype Types	19
2.5	Fidelity Matched to the Question	20
2.5.1	The Effort–Fidelity Model	20
2.5.2	Running Example: Choosing Fidelity for the Sorter	21
2.6	The Documentation Layer	22
	Review Questions	22
	End-of-Chapter Artifact	24
	Chapter Summary	24
3	Specifications & Risk Prioritization	27
3.1	Anatomy of a Specification	27
3.1.1	Functional Requirements	27
3.1.2	Performance Targets	28
3.1.3	Constraints	28
3.1.4	Interfaces	28
3.1.5	Acceptance Criteria	28
3.2	Requirements vs. Design	29
3.2.1	The “What / How” Discipline	29
3.3	Risk-Driven Prioritization	30
3.3.1	The Basic Risk Equation	30
3.3.2	Limitations of the Two-Factor Model	30

3.4	FMEA-Style Ranking (RPN)	31
3.4.1	The RPN Formula	31
3.4.2	Scale Anchors	31
3.5	The Critical Path of Unknowns	32
3.5.1	Identifying the Critical Unknown	32
3.6	Worked Example: IoT Robotic Sorting Subsystem	33
3.6.1	Step 1: Convert Vague Wishes into Measurable Requirements	33
3.6.2	Step 2: Compute RPN for Three Subsystems	34
3.6.3	Step 3: Rank and Interpret	35
3.6.4	Try It Yourself	36
	Review Questions	37
	End-of-Chapter Artifact	37
	Summary	38



CHAPTER 1

OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Every project begins with a claim: *this idea is worth building*. Your job in Project I is to find out whether that claim is true—cheaply, quickly, and with enough documentation that the answer is defensible. By the end of this course you will have built a working proof-of-concept, assembled the record that shows how you got there, and produced the business case that connects your hands-on work to the real world of engineering firms and markets. This chapter gives you the map. You will see what you are building, why it matters, what separates a prototype from a finished product, and what turns a validated prototype into revenue.

Learning Objectives

1. State what a prototype is for and describe the two deliverables you will produce in Project I.
2. Describe how a proven prototype becomes a product and identify the link between Project I and Project II.
3. Explain your role as a technician working alongside engineers.
4. Explain why industry work is justified by the value it creates, and interpret a negative economic finding correctly.
5. Identify the principal business models that turn a validated prototype into revenue.

1.1 What You Will Build

You will build two things: a **proof-of-concept prototype** and **the record of how you got there**.

The prototype is a working artifact that answers a specific technical and business question. It does not need to be pretty, manufacturable, or certifiable. It needs to

2 OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

demonstrate that the core idea functions under realistic conditions.

The record is a portfolio of documents—specification, risk register, architecture diagrams, build logs, test data, and cost analysis—that accumulate as you work. You do not write the portfolio at the end. You assemble it from artifacts you produce at each stage. If you work that way, the portfolio is nearly complete before you realize you are making one.

Together, the prototype and its record constitute your output from Project I. The record is as important as the hardware: an engineer can reproduce, evaluate, and extend documented work; undocumented work is disposable.

1.2 The Course Map: Two Tracks Over Twelve Weeks

Figure 1.1 shows the full engineering journey from concept to market. Project I occupies the second stage—the prototype—and Project II the third, where the proven concept is developed into a manufacturable product. The arrow between them is not automatic: what crosses it is the documentation you create in Project I.

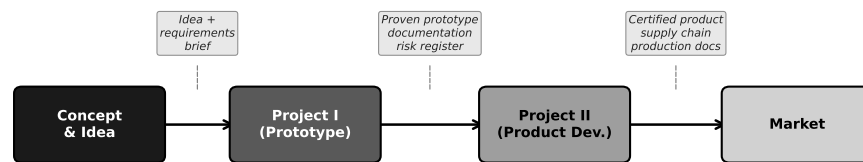


Figure 1.1: The engineering pipeline from concept to market. Project I delivers the proven prototype and its documentation; Project II turns that foundation into a manufacturable product.

The course runs two **parallel tracks**: a theory track and a lab (practice) track. Figure 1.2 shows how they align across the twelve weeks.

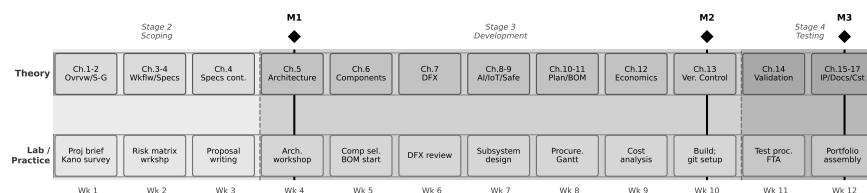


Figure 1.2: The two-track course structure over twelve weeks. The theory track introduces each topic just before the lab requires it; the seven workflow stages unfold across the practice track, with three graded milestones (M1–M3).

The theory track moves through the seventeen chapters of this book at roughly one to two chapters per week, with Chapter 1 and Chapter 2 paired in Week 1. The lab track runs the **seven-stage Prototype Development Cycle**, which you will study in depth in Chapter 3. At the overview level, the seven stages are:

1. **Specification & Risk** — translate the idea into measurable requirements and a

risk ranking.

2. **Scoping & Planning** — decide what you will build, set a schedule, and identify long-lead procurement.
3. **Architecture** — decompose the system into subsystems and define their interfaces.
4. **Subsystem Design** — make detailed design decisions for each subsystem.
5. **Build & Integration** — fabricate and assemble; bring subsystems together.
6. **Validate & Measure** — test against the specification; collect data; decide whether to iterate.
7. **Compile & Economics** — assemble the portfolio and analyse the cost and business case.

Three milestones punctuate the semester. **Milestone 1** (Proposal, end of Week 3) delivers your specification and risk register. **Milestone 2** (Design Review, end of Week 9) delivers your architecture, build records, and preliminary cost analysis. **Milestone 3** (Final Portfolio, end of Week 12) delivers the complete project record.

The theory topics are sequenced so that each week's reading lands just as the lab needs it: you study specifications in Week 3, then immediately write your own in the lab. This is not a coincidence—it is the design of the course.

1.3 From Prototype to Product: The Gap

A prototype and a product solve different problems. A **prototype** answers a question about technical feasibility and business viability. A **product** is an artifact designed to be made reliably, in volume, at a cost that leaves margin, by people who were not involved in the original design, and supported over a service life that may span years.

The gap between them is real and wide. A successful prototype does not automatically become a product. The following properties must be designed in—not bolted on afterward:

Manufacturability. Can the design be made in volume using available processes, tooling, and supply chains? Hand-soldered one-of-a-kind assemblies are not manufacturable at scale.

Reliability. Does the design meet a minimum operating life under realistic field conditions? A prototype that runs for an hour in a lab may fail within a week in the field due to thermal cycling, vibration, or humidity.

Cost at volume. Is the per-unit cost, at the production volumes the business requires, low enough to leave an acceptable margin? Chapter 10 develops the cost model in full.

Certification. Does the design meet applicable electrical safety, radio emissions, and (for IoT and robotics) functional safety standards? Certification is a legal prerequisite in most markets, not an optional finishing step.

Supportability. Can the product be diagnosed and repaired by field technicians? Are replacement parts available over the product lifetime? Is the documentation sufficient for a technician who was not present at the original build?

None of these properties is addressed in Project I. That is not a failure; it is appropriate scope management. Your job in Project I is to answer the central technical and business question as quickly and cheaply as possible. Project II then takes that answer as its starting point and builds the full product around it. What crosses the boundary is your documentation: the specification, the architecture, the risk register, the validated design decisions. The formal process by which a prototype earns the right to become a product — Stage-Gate — is introduced in Chapter 2.

1.3.0.0.1 In Practice. A team building a wearable health monitor completes a functional prototype in twelve weeks: it reads heart rate, displays data on a small screen, and transmits to a phone app. The team is pleased—and surprised, six months later, to learn that the product redesign for manufacturing took an additional eighteen months. The enclosure needed injection-moulding tooling. The radio module required FCC and IC certification. The battery management circuit had to be redesigned for the thermal envelope of a wearable. The prototype proved the concept; the product required a separate, larger effort. Neither effort was wasted.

1.4 Working with Engineers: What You Own

In industry you will work on a team that includes engineers. Understanding who owns what prevents frustration and makes you more effective.

Engineers own:

- Requirements—what the system must do, expressed as measurable criteria.
- Design accountability—the engineer signs off on the architecture and carries legal responsibility for the design.
- Risk decisions—the call to proceed, stop, or redesign rests with the engineer of record.

You own:

- Building—breadboarding, PCB assembly, mechanical fabrication, 3D printing, cable harnesses.
- Testing—executing the test procedure, collecting and recording data honestly, flagging anomalies.
- Documenting—keeping the build log current, maintaining the as-built BOM, photographing assembly steps.
- Procuring—sourcing components, getting quotes, placing purchase orders, receiving and inspecting parts.

- Troubleshooting—diagnosing faults at the bench level.

This boundary is not about status; it reflects training and accountability. A technician who takes over design decisions without the engineer's sign-off creates liability for everyone. A technician who executes, documents, and communicates problems clearly makes the engineer's job tractable. Your hands-on judgment—knowing that a solder joint looks wrong, that a component is getting too hot to touch, that the measured current does not match the schematic—is irreplaceable. Engineers who have lost touch with the bench depend on you for that judgment.

1.5 Engineering Businesses by Size

The same prototype, handed to three firms of different sizes, will be handled very differently. Figure 1.3 illustrates how process formality, documentation weight, and approval layers scale with firm size.

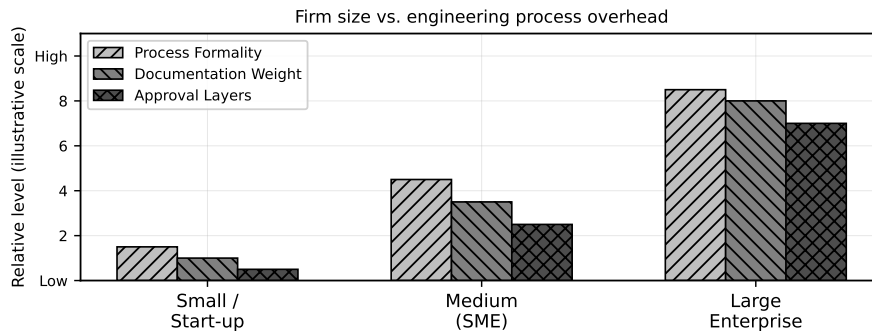


Figure 1.3: Firm-size spectrum: process formality, documentation weight, and approval layers all increase substantially from small start-ups to large enterprises. The prototype development approach must match the firm context.

Small firms and start-ups (under ≈ 20 people) move fast and tolerate ambiguity. A single engineer may own the entire design. Documentation is minimal—often just enough to reproduce the build. The prototype is frequently made by the same people who conceived it. Speed matters more than process. The risk is that institutional knowledge lives in people's heads and disappears when they leave.

Medium firms (SMEs) (20–500 people) have begun to institutionalise. There are defined roles, some formal review gates, and more documentation requirements. A design change needs sign-off from more than one person. The prototype team is likely distinct from the product team, so documentation quality directly affects the handoff.

Large enterprises (500+ people) operate under heavy process—ISO standards, stage-gate reviews, change control boards, formal test plans, and full configuration management. A prototype at a large firm may require months of internal approval before the first component is ordered. Documentation must be complete, traceable, and auditable.

None of these contexts is inherently better. A start-up that adopts enterprise process collapses under bureaucratic overhead. A large firm that abandons process

loses traceability and produces unrepeatable results. Your job is to recognise the context you are in and calibrate your documentation effort accordingly.

1.5.0.0.1 In Practice. A three-person IoT start-up builds a temperature monitoring module in four weeks: breadboard, firmware, cloud logging, mobile app. The “documentation” is a shared folder with photos and a text file. Two years later, the same company has grown to forty people. A new hire tries to reproduce the original circuit for a customer request. The photos are there; the design rationale is not. It takes three weeks to reverse-engineer the component selection. The original four weeks of work left a four-week debt that compounded with every new hire.

1.6 Science for Profit: Cost, Time, and Volume

Engineering work is not science for its own sake—it is science directed at creating value. This does not diminish the intellectual content; it sharpens the questions. In a commercial context, every technical decision carries an economic consequence, and the three most important constraints are **cost**, **time**, and **volume**.

Cost constrains what you can build and at what margin. A solution that is technically elegant but costs twice the target variable cost is not a viable product.

Time constrains competitive advantage. A prototype finished six months late may miss a market window entirely. Time is the one resource you cannot recover.

Volume determines the economic structure. At low volumes, fixed costs (tooling, engineering, certification) dominate the per-unit cost. At high volumes, variable costs dominate. Chapter 10 develops this model formally; here, it is enough to recognise that the prototype’s per-unit cost—often assembled by hand from individual components—tells you nothing about what the product will cost at 1,000 units.

A critical professional skill is the ability to interpret a negative economic finding correctly. Suppose your prototype proves that the design works but the cost analysis shows that no viable margin exists at any realistic production volume. This is not failure—it is the most valuable result the prototype could have produced. You have saved the firm the cost of a full product development effort on an unviable design. Document the finding clearly, state the assumptions, and let the engineering team decide whether to pivot or stop. A prototype that says “no” is doing exactly what a prototype is for.

1.7 How a Prototype Becomes Revenue

Validating a prototype is not the end of the story. The business question that drives the prototype is always, ultimately: how does this create value that someone will pay for? Figure 1.4 maps the major revenue models available once a prototype has been validated.

Product sale (direct). The most familiar model: manufacture a unit, sell it for

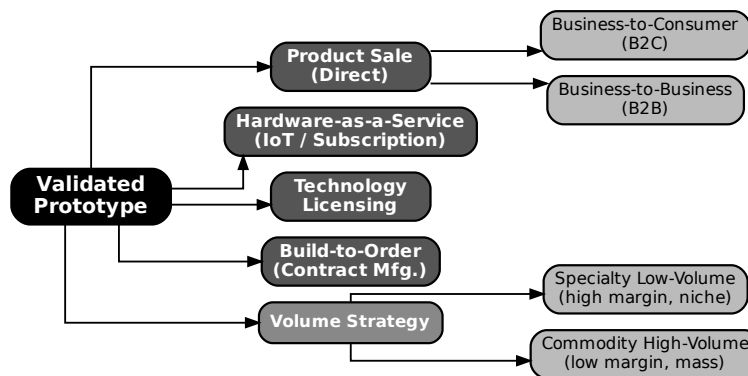


Figure 1.4: Revenue models reachable from a validated prototype. The choice of model shapes what the prototype must prove and what the product development effort must deliver.

more than it costs to make and sell. The two dominant sub-types differ in who the customer is. In a **business-to-consumer (B2C)** model, you sell to individuals; volume is potentially high but margins are typically lower and customer acquisition is expensive. In a **business-to-business (B2B)** model, you sell to companies; volumes are lower but unit prices and margins are generally higher, and relationships carry more weight than brand.

Hardware-as-a-Service (HaaS / IoT subscription). The customer pays an upfront hardware fee and a recurring subscription for connectivity, data processing, and support. The prototype must prove both the hardware function *and* the cloud-connectivity architecture. This model creates recurring revenue and customer lock-in but requires ongoing operational cost for the cloud backend. It is common for IoT devices that produce data the customer values continuously.

Technology licensing. The firm does not manufacture; it licenses the design, firmware, or algorithm to other manufacturers. The prototype proves that the intellectual property is real and valuable. Licensing requires strong IP documentation and often patents. Beyond licensing, intellectual property ownership is a business-model consideration for every revenue model — Chapter 15 covers the practical IP obligations a technician must understand.

Build-to-order (contract manufacturing). The firm builds custom units against specific customer purchase orders. Volume is low per customer; the customer drives the specification. Margins can be high because the customer is paying for customisation. The prototype is typically a demonstration to win the contract.

Volume strategy: specialty vs. commodity. Within product sale, the firm chooses where on the volume spectrum to compete. **Specialty low-volume** production targets niche markets with high unit prices and high margins; low volume means less pressure on tooling costs but fewer units over which to amortise fixed costs. **Commodity high-volume** production targets large markets with low unit prices and thin margins; every cent of variable cost matters, and design-for-manufacture is mandatory from the first design decision.

The choice of revenue model is made before the prototype is designed, not after. It shapes what the prototype must prove, what fidelity is needed, and what documentation must be produced. A HaaS prototype that lacks a working cloud connection has not answered the central business question. A licensing prototype with no IP documentation has nothing to license.

Worked Example: The Sorting Subsystem Business Case

Throughout this book, every chapter works through the same running example: a **connected IoT robotic sorting subsystem**—a sensor-guided actuator that classifies items on a bench conveyor, diverts them to the correct output lane, reports status over a network, and is designed as a module within a larger automation product.

Firm context. The team is a twelve-person engineering start-up that has identified a market in light-industrial sorting for food processing and pharmaceutical packaging. The firm is too small for enterprise process but large enough to have separate engineering and operations functions. Documentation weight is medium: enough to enable reproduction by another team member, not enough to require formal configuration management.

Business model. The chosen model is **Hardware-as-a-Service**: customers purchase the sorter module at a hardware price and pay an annual subscription for cloud-hosted analytics, remote monitoring, and firmware updates. This model requires the prototype to prove two things: (1) the mechanical and electronic sorter works reliably, and (2) the cloud connectivity architecture is sound and can sustain the required data throughput.

What the prototype must answer. The central question is: *Can a sub-\$500 CAD bill-of-materials sorter module achieve $\geq 95\%$ sort accuracy at 6 items/min while maintaining a reliable MQTT connection over a standard Wi-Fi network?* If yes, the hardware is viable and the HaaS model can proceed. If no, the firm needs to know whether the failure is in the mechanical design, the sensing, the firmware, or the network layer—so the portfolio must be structured to show that.

Margin calculation. Suppose the firm targets a hardware unit price of $p = \$1,200$ CAD and estimates the variable cost of a production unit (materials + direct labour at volume) at $c_{\text{var}} = \$480$ CAD. (These are early estimates; Chapter 10 will refine them using the full cost model.)

The unit margin is:

$$m = p - c_{\text{var}} = \$1,200 - \$480 = \$720 \text{ CAD/unit.} \quad (1.1)$$

The gross margin fraction is:

$$\text{GM} = \frac{p - c_{\text{var}}}{p} = \frac{\$720}{\$1,200} = 0.60 \quad (60\%). \quad (1.2)$$

A 60% hardware gross margin is strong for an electromechanical module. It must, however, sustain both the hardware warranty cost and the ongoing cost of the cloud subscription infrastructure. Chapter 10 will introduce the full unit-cost model,

$C_{\text{unit}}(N) = C_{\text{fixed}}/N + c_{\text{var}}$, which shows how the fixed non-recurring costs (tooling, firmware development, certification) dilute this margin at low production volumes.

Project context brief (your first portfolio artifact).

- *Project*: IoT robotic sorting subsystem module.
- *Business model*: Hardware-as-a-Service; hardware sale + annual cloud subscription.
- *Firm context*: 12-person engineering start-up; medium documentation weight.
- *Business question the prototype must answer*: Can the module achieve $\geq 95\%$ sort accuracy at 6 items/min with a reliable cloud connection, at a BOM cost below \$500 CAD?

Try It Yourself

A micro-robotics start-up is developing a vision-guided pick-and-place module. They plan to sell it directly to electronics assembly firms (B2B product sale). Their target unit price is $p = \$950$ CAD and their estimated variable cost is $c_{\text{var}} = \$285$ CAD.

1. Compute the unit margin $m = p - c_{\text{var}}$.
2. Compute the gross margin fraction $\text{GM} = (p - c_{\text{var}})/p$ and express it as a percentage.
3. Using Figure 1.4, identify which revenue model applies and state one thing the prototype must prove to make that model viable.

(Answers not provided — work through the arithmetic before moving on.)

1.7.0.0.1 Common Pitfalls. Technically impressive $\neq$ good prototype. A prototype that demonstrates six different features but answers none of the business questions clearly is not a good prototype. Novelty and complexity are not success criteria. The success criterion is: *did you answer the question?*

"It works" $\neq$ "the business case is proven." A prototype that demonstrates function but carries a BOM cost of \$1,400 CAD against a \$1,200 target price is not a viable product, regardless of how well it works. The economic analysis is not an afterthought—it is half of the prototype's output.

Underrating the technician's judgment. Engineers rely on your observation that the actuator is running hotter than expected, that the network connection drops in the presence of the motor drive, or that a component you received does not match the datasheet. These observations, documented and communicated promptly, are the inputs that prevent costly mistakes. Do not wait to be asked.

Reading a negative economic finding as failure. If the cost analysis shows that the HaaS model is not viable at any realistic volume, that result should be reported clearly, not buried or explained away. The firm can decide to pivot—to a different revenue model, a different component set, or a different market. It cannot decide anything if the analysis is suppressed.

Review Questions

1. **(Conceptual)** A product must satisfy five properties that a prototype does not. List all five and write one sentence explaining what each requires in practice.
2. **(Conceptual)** The chapter states that “a prototype that returns a negative economic result is not a failure.” Explain this claim in your own words. What should the team do with the finding, and why is suppressing it dangerous?
3. **(Applied — calculation)** A start-up prices a wireless sensor node at $p = \$340$ CAD and initially estimates $c_{\text{var}} = \$136$ CAD.
 - (a) Compute m and GM%.
 - (b) After a detailed BOM review, the true variable cost is $c_{\text{var}} = \$204$ CAD. Recompute m and GM%.
 - (c) By how many percentage points did the gross margin change? Comment on whether the product remains financially attractive.
4. **(Applied — classification)** A company has developed a machine-learning algorithm that identifies crop disease from smartphone photos. They plan to license it to agricultural equipment manufacturers. Using Figure 1.4 and the definitions in §1.7, identify the revenue model and state two things the prototype must demonstrate to support that model.
5. **(Applied — multi-step)** A proof-of-concept prototype costs \$2,400 CAD to build. The team estimates that the eventual product will sell at $p = \$600$ CAD with $c_{\text{var}} = \$180$ CAD.
 - (a) Compute m and GM%.
 - (b) The \$2,400 build cost is *not* c_{var} . Name the cost category it belongs to, and explain why it does not appear in the margin formula at this stage. (*Hint: Chapter 10 will introduce the full cost model.*)

End-of-Chapter Artifact: Project Context Brief

By the end of Week 1, you will produce a one-page **project context brief** for your own project. It is the first document in your portfolio and frames everything that follows. It must contain:

1. A description of the project (one paragraph, no jargon).
2. The chosen business model, with a one-sentence justification.
3. The firm context (size, documentation standard, your role).
4. The single business question the prototype must help answer.
5. A preliminary margin estimate: state p , c_{var} , compute m and GM%, and note the assumptions behind c_{var} .

Keep the brief to one page. Its purpose is not comprehensiveness but clarity: after reading it, a second team member should be able to say whether the project is technically and commercially plausible at a first glance.

Summary

Project I delivers two things: a proof-of-concept prototype and the portfolio that documents how you built and tested it. The prototype's job is to answer one specific technical and business question—not to be a miniature product. Between Project I and Project II lies a deliberate gap: manufacturability, reliability, cost at volume, certification, and supportability must all be designed into the product in Project II, using the prototype's answers as a foundation.

As a technician, you own the build, the test, the documentation, and the procurement. Your hands-on judgment is the most direct input the engineering team has to reality.

Firms handle prototypes differently depending on their size: start-ups move fast with minimal process; enterprises move carefully with heavy documentation. Calibrate your effort to the context.

The economic constraints—cost, time, and volume—are design variables, not external obstacles. A prototype that returns a negative economic result is not a failure; it is evidence. The revenue model chosen before the prototype is built shapes everything: what the prototype must prove, what fidelity is required, and what documentation must be produced.

Up next: Chapter 2 establishes the formal product development process — the Cooper Stage-Gate model and Kano analysis — that gives every subsequent chapter its professional context. You will see where this course and Project II sit within that model, and how Kano analysis determines what the prototype must prove.



CHAPTER 2

THE PROTOTYPING MINDSET & THE WORKFLOW

2.0.0.0.1 Where this fits. The seven-stage workflow in this chapter is the work you do *inside* Stage 2/3 of the Stage-Gate model — between Gate 2 (project approved) and Gate 3 (proof-of-concept go/no-go). Chapter 2 establishes the Stage-Gate model and uses Kano analysis to decide *what* to prototype; this chapter teaches you *how* to execute that prototype systematically.

Chapter 1 established the business context: what a prototype must prove, and why that question must be answered before any product development investment is justified. This chapter gives you the process that answers it.

Every prototype you build costs time, materials, and opportunity. Build the wrong one — too early, too polished, or targeting the wrong question — and you burn weeks that a one-day breadboard experiment would have saved. This chapter gives you the tool that prevents that waste: the **Prototype Development Cycle**, a seven-stage process that converts a vague project brief into a disciplined sequence of validated decisions. By the end of the chapter you will be able to plan a prototype, defend your fidelity choice with arithmetic, and hand a concise question statement to any reviewer, client, or collaborator.

Learning Objectives.

1. Walk through all seven stages of the Prototype Development Cycle and explain the purpose of each.
2. Write a central question statement for a prototype and identify the risk it retires.
3. Apply the controlled Design ↔ Validate iteration loop correctly.
4. Select the appropriate prototype type for a given engineering question.
5. Compute the relative effort ratio between two fidelity options using $E(f) = E_0 \cdot k^f$ and use the result to justify a fidelity choice.

2.1 Prototype = Answer to a Question, Reducer of Risk

A prototype is not a small version of the final product. It is not a demonstration for a trade show, a proof that your team is making progress, or practice for the real build. A prototype is a **minimum sufficient artifact** constructed to answer one specific technical or business question and to retire a corresponding risk.

Write the question down before you pick up a soldering iron. If you cannot write it in a single sentence, you do not yet know what you are building. Consider the difference between these two framings:

- *Vague*: “We need to build a prototype of the sorter.”
- *Precise*: “Can a sub-\$500 CAD BOM sorter module achieve $\geq 95\%$ sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?”

The precise framing tells you exactly what to build (a low-cost sorter mechanism with Wi-Fi MQTT), what to measure (accuracy at a fixed throughput rate), and what threshold constitutes a “yes” answer. It also tells you what *not* to build: an injection-moulded housing, CE-marked power supply, or production PCB layout are irrelevant until the accuracy and connectivity question is answered.

Risk in this context is the probability that an unverified assumption collapses the project. Every assumption you have not tested is a risk. A prototype converts an assumption into a measurement, and a measurement into a decision. Figure 2.1 shows how even a low-fidelity prototype delivers substantial intrinsic value — knowledge and risk reduction — long before artifact quality approaches production standards.

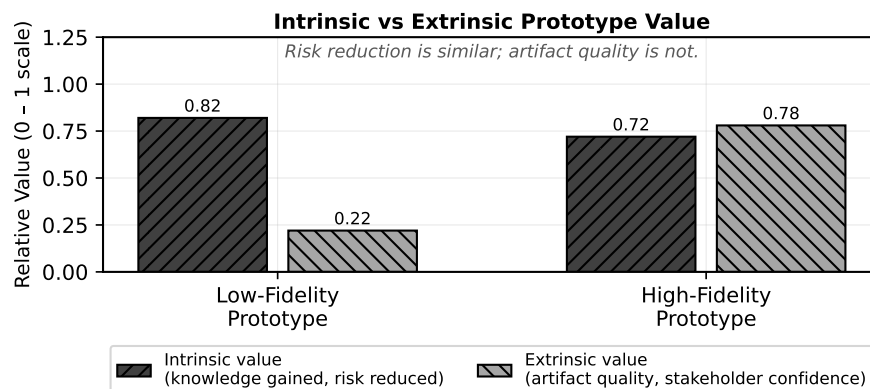


Figure 2.1: Intrinsic value (knowledge, risk reduction) is substantial even at low fidelity; extrinsic value (artifact quality, stakeholder confidence) grows mainly at high fidelity. Build to the fidelity the question requires, not the fidelity that impresses.

A prototype delivers two kinds of value. **Intrinsic value** is the knowledge gained and the risk retired by answering the central question; it is high even at low fidelity, because the question does not require a polished artifact. **Extrinsic value** is the quality and completeness of the artifact itself — its appearance, robustness, and the confidence it gives to external stakeholders. Extrinsic value grows mainly at higher fidelity levels because it depends on how finished the prototype looks and

behaves, not on whether the central question is answered. Build to the fidelity that delivers the intrinsic value you need; extrinsic value follows in later milestones.

Framing every build as an answer to a question keeps the team aligned and prevents the most common prototyping failure: building features before the central question is answered.

2.2 The Seven Stages

Figure 2.2 shows the full Prototype Development Cycle. The seven stages run left to right; Stages 4 through 6 form the core build-test loop that iterates until the central question is answered. The loop is controlled — you exit it on a criterion, not when you run out of time.

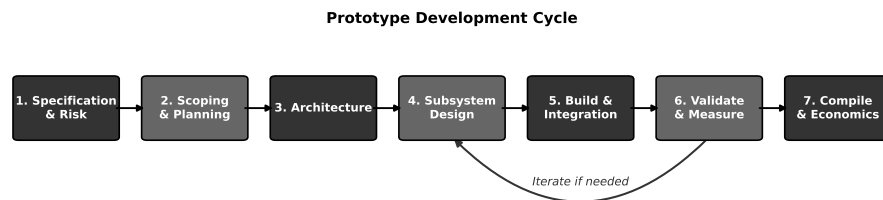


Figure 2.2: The seven-stage Prototype Development Cycle with the Design ↔ Validate iteration loop. Stages 4 through 6 form the core build-test loop; iteration is controlled, not ad-hoc.

2.2.1 Stage 1: Specification & Risk

Stage 1 transforms the project brief into a structured set of **specifications** and a ranked list of **risks**. You are not designing anything yet. You are reading, asking questions, and writing.

A specification at this stage has four components:

1. *Functional requirements* — what the system must do (“sort at ≥ 6 items/min”).
2. *Performance bounds* — measurable thresholds (“ $\geq 95\%$ accuracy”, “MQTT round-trip ≤ 200 ms”).
3. *Constraints* — non-negotiable limits (“BOM cost $\leq \$500$ CAD”, “fits on a 600 mm bench conveyor”).
4. *Assumptions* — things believed to be true but not yet measured.

Every assumption in item 4 is a risk candidate. You rank risks by the product of probability and impact, and you identify which prototype stage will retire each one. By the end of Stage 1 you have a written specification document and a risk register. These are the two artifacts that every subsequent stage references.

2.2.1.0.1 In Practice. A one-hour specification session with your full team at the whiteboard saves more time than a week of uncoordinated building. Force every team member to write at least one assumption. Assumptions that no one writes down are the ones that kill projects in Week 10.

2.2.2 Stage 2: Scoping & Planning

Stage 2 converts the risk register into a prototype plan. You decide: which risks must be retired by this prototype, what fidelity level is sufficient to retire them, how long the build will take, and what success looks like.

The key output is a **prototype plan** with:

- The central question (one sentence).
- The chosen fidelity level and justification.
- A work-breakdown with task ownership and duration.
- Explicit pass/fail criteria for the validation step.

Scoping means saying “not this prototype.” If the question is about sort accuracy, the housing material is out of scope. Resist the urge to answer every question at once; you will get a chance in the next milestone. A well-scoped prototype plan is a document your supervisor can review in ten minutes and approve or correct before your team spends any build time — and it is the key output that justifies a Gate 2 approval; Chapter 4 develops it into a full specification and risk register.

2.2.3 Stage 3: Architecture

Stage 3 partitions the system into **subsystems** with defined interfaces. You are not selecting components yet; you are drawing boundaries.

A useful architecture diagram shows:

- Each subsystem as a labelled block.
- Every signal and power interface as an arrow with direction and type (digital, analog, mechanical, protocol name).
- The measurement points where validation data will be collected.

The architecture is the contract between team members. If two people are building adjacent subsystems without an agreed interface, integration will fail. In Stage 3 you negotiate and document those contracts before anyone starts detailed design.

The architecture also determines which subsystem risks can be tested independently. If sort accuracy depends on both the sensor subsystem and the conveyor speed controller, you cannot test one without the other — and your plan must reflect that coupling.

2.2.4 Stage 4: Subsystem Design

Stage 4 is detailed design: component selection, schematic capture, firmware architecture, mechanical part design, and BOM costing. This is where your prior coursework in PCB design, embedded programming, and mechanical CAD becomes the primary activity.

In a prototype context, Stage 4 output does not need to be production-ready. A breadboard schematic is a legitimate Stage 4 artifact if it enables the Stage 5 build to proceed and the Stage 6 measurement to be valid. Keep every design decision traceable to a specification line or a risk item. If you cannot trace it, question whether you need it.

Stage 4 is the entry point to the Design $\leftrightarrow$ Validate loop described in Section 2.3. You design at the subsystem level, build the subsystem, measure it, and return to design if the measurement fails — before integrating with adjacent subsystems.

2.2.5 Stage 5: Build & Integration

Stage 5 is fabrication and assembly: soldering, 3D printing, firmware flashing, wiring, and bringing subsystems together at their defined interfaces.

Integration is a distinct activity from subsystem build. Subsystems that each pass their individual tests routinely fail at integration because interface assumptions were wrong. Run integration in a defined order: bring up the simplest subsystem first, verify its standalone behaviour, then attach the next subsystem and re-verify the combined behaviour before adding the third.

Document every deviation from the Stage 4 design as it happens; post-hoc reconstruction from memory is unreliable and undermines the entire design stage.

2.2.5.0.1 In Practice. Keep a shared build log — a shared document or a lab notebook photographed daily — recording what was built, what changed from the design, and any anomalies observed. The build log is the evidence that your Stage 6 validation measurements are trustworthy.

2.2.6 Stage 6: Validate & Measure

Stage 6 is where you answer the question you wrote in Stage 2. **Validation** is a structured measurement campaign against the pass/fail criteria defined before you built anything.

A valid measurement has:

- A defined test procedure that a second engineer can reproduce.
- Sufficient repetitions to estimate uncertainty (minimum 10 trials for a binary outcome like “sort correct/incorrect”).

- Documented environmental conditions (supply voltage, temperature, Wi-Fi signal strength, conveyor belt load).
- A recorded outcome: pass, fail, or inconclusive with reason.

If the outcome is “pass,” you exit the loop and proceed to Stage 7. If the outcome is “fail,” you return to Stage 4 with a specific design change hypothesis, not a vague intention to “try something different.” This controlled return is what distinguishes disciplined iteration from thrashing.

2.2.7 Stage 7: Compile & Economics

Stage 7 closes the prototype milestone. You compile the evidence: test results, BOM cost actuals, build time log, and a comparison of achieved performance against specification. You then assess the economic implications.

For a commercial project, Stage 7 includes a preliminary **unit economics** analysis: does the validated BOM cost, at the production volume assumed in the business model, support the target margin? For the running example — a 12-person start-up with a Hardware-as-a-Service model, price $p = \$1,200$ CAD, and variable cost $c_{\text{var}} = \$480$ CAD — the contribution margin per unit is:

$$m = p - c_{\text{var}} = \$1,200 - \$480 = \$720 \text{ CAD} \quad (2.1)$$

If the BOM comes in above \$500 CAD target, Stage 7 flags that the economics are compromised before the team invests in Milestone 2 tooling. Stage 7 also produces the **go/no-go recommendation** for the next milestone and a list of risks that remain open. This recommendation is the prototype-evidence component of the Gate 3 decision: the project proceeds to Stage 3 development only if the Stage 7 compilation supports it.

2.3 The Controlled Iteration Loop (Design ↔ Validate)

Stages 4, 5, and 6 form a loop. The loop is necessary because prototypes reveal information that design alone cannot predict: a sensor saturates at lower ambient light than the datasheet implied, firmware interrupt latency is 40% higher than estimated, a 3D-printed bracket flexes under motor torque. Iteration is not a sign of poor planning; it is the mechanism by which prototyping generates knowledge.

What must be controlled is *how* you iterate.

Uncontrolled iteration looks like this: the test fails, someone changes three things simultaneously, the next test passes, and no one knows which change fixed the problem. This is common; it is also useless as engineering knowledge, because you cannot reproduce the fix reliably or explain it to a manufacturing partner.

Controlled iteration imposes two rules:

1. *One variable at a time.* When a measurement fails, form a hypothesis about the cause, change exactly one design element, and re-measure. If you must

change two things to make the system testable, document the dependency explicitly.

2. *Exit on a criterion, not on a deadline.* Define in Stage 2 what “good enough” means. Exit the loop when the measurement meets that criterion. Do not exit because the schedule says so without recording which criteria remain unmet.

Every pass through the loop should reduce uncertainty by answering a smaller sub-question. If you cannot name the sub-question before you change something, you are not iterating — you are guessing.

2.3.0.0.1 In Practice. Limit yourself to three full loop passes before calling a team checkpoint. If after three passes the central question is not answered, the problem is likely in the scope, the architecture, or the specification — not in the build. Step back to Stage 2 or Stage 3 rather than continuing to iterate at Stage 4.

2.4 Prototype Types

Not all prototypes serve the same purpose. Selecting the right type for your question prevents over-building and keeps the team focused. The four primary types used in engineering product development are:

Proof-of-Concept (PoC) Answers a binary feasibility question: can the physics work? Appearance is irrelevant. A PoC uses whatever materials are fastest to assemble — breadboards, hook-up wire, 3D-printed brackets, off-the-shelf modules. Target fidelity: Level 1 or 2.

Functional prototype Demonstrates that the system performs its core function in representative conditions. Appearance may still be a mock-up. Firmware is real but not hardened. Used to answer “does it work under realistic operating conditions?” Target fidelity: Level 3.

Looks-like/works-like prototype Combines representative form and full function. Housing geometry, interface locations, and weight approach final product. Used to answer “is the user experience acceptable?” and “can we hit the cost target?” Target fidelity: Level 4.

Pre-production prototype Built from production-intent processes: PCB from the chosen contract manufacturer, housing from production tooling, firmware at release candidate version. Answers: “will this survive the factory and the field?” Target fidelity: Level 5.

Choosing a looks-like/works-like prototype to answer a feasibility question is the most common and most expensive mistake in early-stage product development. Match the type to the question first; fidelity follows from type.

2.5 Fidelity Matched to the Question

Fidelity is the degree to which a prototype resembles and performs like the final product. Higher fidelity costs more. The cost growth is not linear.

Figure 2.3 shows relative effort plotted against fidelity level. The relationship is approximately exponential.

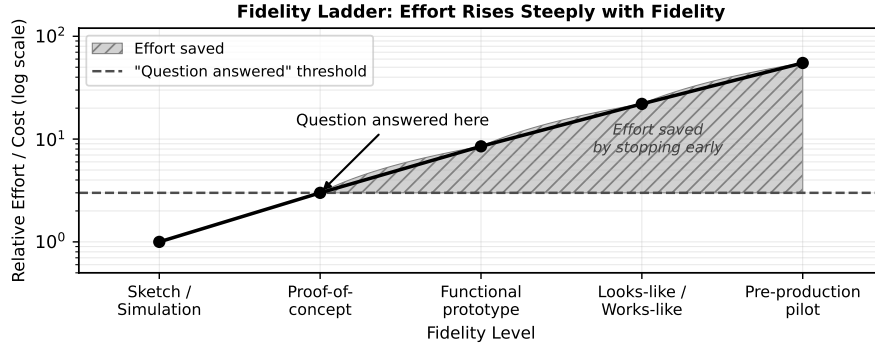


Figure 2.3: Relative effort rises steeply with fidelity level. A proof-of-concept (Level 2) answers many questions at a fraction of the cost of a looks-like/works-like prototype (Level 4).

2.5.1 The Effort–Fidelity Model

Let $f \in \{1, 2, 3, 4, 5\}$ be the fidelity level, where $f = 1$ is a hand sketch or simulation and $f = 5$ is a pre-production prototype built with production-intent processes. Define **relative effort** $E(f)$ as:

$$E(f) = E_0 \cdot k^f, \quad k > 1 \quad (2.2)$$

where E_0 is a baseline effort constant and k is the per-level effort multiplier. The model captures the empirical observation that each fidelity step requires roughly k times the total effort of the step below it, because higher fidelity demands tighter tolerances, more sophisticated tooling, more thorough testing, and more documentation.

2.5.1.1 Effort Table

Set $E_0 = 1$ and $k = 2.5$ as representative values. Table 2.1 shows the resulting effort at each level.

The ratio of effort at $f = 5$ relative to $f = 2$ is:

$$\frac{E(5)}{E(2)} = \frac{k^5}{k^2} = k^3 = 2.5^3 = 15.6 \quad (2.3)$$

If your question is answered at $f = 2$, building to $f = 5$ multiplies your effort by 15.6×. Equivalently, $\frac{E(5) - E(2)}{E(5)} \approx 93\%$ of the total $f = 5$ effort is *spent after the question is already answered*. That is the waste the fidelity discipline prevents.

Table 2.1: Relative effort $E(f) = 2.5^f$ for fidelity levels 1 through 5.

Level f	Type	Relative Effort $E(f)$
1	Sketch / simulation	$2.5^1 = 2.5$
2	Proof-of-concept	$2.5^2 = 6.25$
3	Functional prototype	$2.5^3 = 15.6$
4	Looks-like/works-like	$2.5^4 = 39.1$
5	Pre-production	$2.5^5 = 97.7$

2.5.2 Running Example: Choosing Fidelity for the Sorter

Return to the central question for the connected IoT robotic sorting subsystem:

“Can a sub-\$500 CAD BOM sorter module achieve $\geq 95\%$ sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?”

The team is deciding between two prototype options:

Option A — Looks-like/works-like ($f = 4$): Custom PCB with production layout, injection-moulded housing mock-up, full production firmware with over-the-air update support. Estimated build time: 14 weeks.

Option B — Proof-of-concept ($f = 2$): Breadboard electronics, 3D-printed mounting bracket, firmware with sort accuracy event logging, Wi-Fi MQTT connectivity test. Estimated build time: 3 weeks.

Applying the effort model with $k = 2.5$:

$$E(4) = k^4 = 2.5^4 = 39.1 \quad (\text{Option A effort})$$

$$E(2) = k^2 = 2.5^2 = 6.25 \quad (\text{Option B effort})$$

$$\frac{E(4)}{E(2)} = k^{4-2} = 2.5^2 = 6.25 \quad (2.4)$$

The fraction of Option A’s effort consumed *after* the question is already answerable at Level 2 is:

$$\frac{E(4) - E(2)}{E(4)} = \frac{39.1 - 6.25}{39.1} \approx 0.84 \quad (84\%). \quad (2.5)$$

If a team’s build estimate is T weeks for Option A, Option B requires $T \times E(2)/E(4) = T/6.25$ weeks. For the 14-week Option A estimate: $14/6.25 \approx 2.2$ weeks — close to the stated 3-week Option B estimate (the difference reflects real project overhead not captured by the model).

Option A requires $6.25 \times$ the effort of Option B. Now ask: does the central question require $f = 4$? The question asks about sort accuracy, throughput rate, BOM cost, and MQTT connectivity. None of those require an injection-moulded housing or a production PCB. A breadboard can deliver accurate sensor readings; a 3D-printed bracket can hold the actuator at the required geometry; a Wi-Fi module on a breadboard can carry MQTT traffic.

The decision is Option B. If Option B answers “yes,” the team proceeds to Milestone 2 with the validated architecture, and Option A becomes the correct investment at that stage. If Option B answers “no” — the sort accuracy is 78% at 6 items/min, or the MQTT latency is unacceptable on a congested network — the team has lost 3 weeks, not 14. The cost of a wrong hypothesis is bounded by the fidelity choice.

2.6 The Documentation Layer

Every stage of the Prototype Development Cycle produces a document. This is not administrative overhead — it is the mechanism by which the prototype generates knowledge that survives beyond the build. A result that is not recorded is a result that will be repeated, at full cost, by the next engineer who faces the same question.

Stage 1 produces the specification and risk register. **Stage 2** produces the prototype plan with pass/fail criteria. **Stage 3** produces the architecture diagram and interface table. **Stage 4** produces schematics, firmware architecture notes, and the BOM; any deviation from the plan is noted here. **Stage 5** produces the build log: what was assembled, what changed from Stage 4, and photographs of key assembly steps. **Stage 6** produces the test procedure, raw data table, and the pass/fail verdict with reasoning. **Stage 7** produces the portfolio entry: test results, cost actuals, and the go/no-go recommendation.

The documentation is not assembled at the end of the milestone. It is produced at each stage, incrementally, so that the portfolio at Stage 7 requires only compilation, not reconstruction from memory. If you find yourself writing Stage 1 documentation in Week 11, the process has already failed. The portfolio you submit at Milestone 3 is exactly the set of documents you produced at each stage — nothing more, nothing less.

2.6.0.0.1 Common Pitfalls.

- **“Prototype = mini-product.”** Building all product features when only one needs to be proved is the most common and most expensive prototyping error. Injection moulding, CE certification, and production firmware are irrelevant at Stage 4 if the question is about sort accuracy. Every feature you build beyond what the question requires is pure waste until the question is answered.
- **“Higher fidelity is always better.”** Fidelity costs effort exponentially, as shown in Table 2.1. A higher-fidelity prototype that cannot yet answer the question delivers zero additional information — it only consumes schedule and budget. Match fidelity to the question; resist the instinct to impress.

Review Questions

1. **(Conceptual)** List the seven stages of the Prototype Development Cycle in order. For Stage 1 and Stage 6, state the primary output that stage produces.

2. **(Conceptual)** Distinguish **intrinsic value** from **extrinsic value** in the context of a prototype. Which type does a proof-of-concept maximise, and why does that make it appropriate for answering a central technical question?
3. **(Applied — calculation)** Using $E(f) = E_0 \cdot k^f$ with $E_0 = 1$ and $k = 3.0$:
 - (a) Compute $E(f)$ for $f = 1, 2, 3, 4, 5$.
 - (b) What is the ratio $E(4)/E(2)$?
 - (c) If a question is answered at $f = 2$, what fraction of the Level-4 effort is saved?
4. **(Applied — decision)** A team must verify that a custom MEMS pressure sensor achieves ± 0.5 Pa accuracy when integrated with their MCU at -20°C .
 - (a) Identify the most appropriate prototype type from §3.4.
 - (b) State the fidelity level from §3.5 and justify the choice in one sentence.
 - (c) If the PoC result is inconclusive — accuracy is within spec at some temperatures but not others — describe one controlled iteration step you would take at Stage 4 before changing the fidelity level.
5. **(Applied — multi-step)** A team estimates 20 weeks to build a pre-production prototype ($f = 5$) at a weekly cost of \$3,500 CAD. Using $k = 2.5$, $E_0 = 1$:
 - (a) Compute $E(2)$ and $E(5)$.
 - (b) What fraction of the 20-week effort corresponds to $f = 2$?
 - (c) How many weeks does the PoC take (round to one decimal place)?
 - (d) How much money is saved by stopping at $f = 2$ if the question is answered there?

Try It Yourself

A firmware team must choose between two prototype options to validate their custom RTOS scheduler under worst-case interrupt load. Both options use $E_0 = 1$ and $k = 3.0$.

Option X — Proof-of-concept ($f = 2$): A minimal test harness running the scheduler on target hardware with synthetic interrupt injection. Estimated build time: 2 weeks.

Option Y — Functional prototype ($f = 3$): Full firmware stack with all application tasks active, running the complete RTOS configuration. Estimated build time: 9 weeks.

1. Compute $E(2)$ and $E(3)$ using $E(f) = E_0 \cdot k^f$.
2. Find the effort ratio $E(3)/E(2)$.
3. Compute the fraction of Option Y effort consumed after the question is already answerable at $f = 2$ (use the formula from Equation 2.5).
4. The central question is: “Does the scheduler meet the 1 ms worst-case interrupt latency under full task load?” State in one sentence whether Option X or Option Y is the correct choice and why, referencing the effort ratio.

(No answer key provided.)

End-of-Chapter Artifact

The deliverable for this chapter is a single written paragraph, submitted to your portfolio, stating: (1) the central question your Milestone 1 prototype will answer, (2) the fidelity level you have chosen, and (3) a justification for that choice in terms of the question, the risk it retires, and the effort saved relative to the next higher fidelity level. This statement is your Milestone 1 prototype scope; submit it as the prototype-evidence component of your Gate 2 package in Week 4.

Running-Example Statement (model your own on this):

The Milestone 1 prototype for the IoT robotic sorting subsystem will answer the following central question: *“Can a sub-\$500 CAD BOM sorter module achieve $\geq 95\%$ sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?”* The chosen fidelity is Level 2 (proof-of-concept), implemented as breadboard electronics, a 3D-printed mechanical mounting bracket, and firmware with sort-event data logging and Wi-Fi MQTT connectivity. This fidelity is sufficient because the question is about sensor accuracy, actuator throughput, and network protocol performance — none of which require a production PCB layout or an enclosure. Advancing to Level 4 at this stage would multiply effort by $6.25\times$ (Equation 2.4) before the feasibility of the core mechanism is established. If the Level 2 prototype answers “yes,” Level 4 is the justified next investment; if “no,” the team reframes the architecture having lost 3 weeks rather than 14.

This statement feeds directly into the specification document you will develop in Chapter 4. Keep it in your portfolio; you will reference and refine it at each subsequent milestone.

Chapter Summary

A prototype is the minimum sufficient artifact that answers a specific technical or business question and retires a corresponding risk. The Prototype Development Cycle organises that work into seven stages: Stage 1 (Specification & Risk) establishes what must be true and what is assumed; Stage 2 (Scoping & Planning) sets the question, the fidelity, and the success criterion; Stage 3 (Architecture) partitions the system into subsystems with defined interfaces; Stage 4 (Subsystem Design) produces the design artefacts; Stage 5 (Build & Integration) realises and assembles those designs; Stage 6 (Validate & Measure) answers the question against pre-defined criteria; and Stage 7 (Compile & Economics) closes the milestone with evidence and a go/no-go recommendation.

Stages 4 through 6 form a controlled iteration loop. Each pass tests one hypothesis, changes one variable, and exits on a measured criterion — not on a deadline and not by accumulated intuition.

Fidelity is not a virtue. It is a variable you set to the minimum level that makes the central question answerable. The effort–fidelity relationship is exponential ($E(f) =$

$E_0 \cdot k^f$), so every unnecessary fidelity level you add multiplies your cost by k before you have an answer. For the sorter running example, choosing Level 2 over Level 4 saves $6.25\times$ the effort on a question whose answer is not yet known.

Transition to Chapter 4. Chapter 4 turns the central question you have just defined into a complete specification and risk register. You will write measurable, testable requirements, separate “what” from “how,” and rank risks using the FMEA risk-priority number — producing the core deliverable of Stage-Gate Stage 2 and the foundation for every subsequent design decision.



CHAPTER 3

SPECIFICATIONS & RISK PRIORITIZATION

Chapter 3 produced two things: a precisely framed central question and a justified fidelity choice. Those outputs identify *what* must be true for the prototype to succeed, but they do not yet say *how true* is true enough — that gap is what a specification fills. This chapter shows how to convert that central question into measurable, testable requirements and a ranked risk register — the Stage-Gate Stage 2 Scoping deliverables — by using two complementary tools: a simple risk equation and an FMEA-style Risk Priority Number.

By the end of this chapter you will be able to:

1. write measurable, testable requirements that can unambiguously pass or fail;
2. separate requirements (the *what*) from design decisions (the *how*); and
3. build and defend a risk-priority ranking that guides prototype focus.

3.1 Anatomy of a Specification

A specification is a contract between the engineering team and every stakeholder who must eventually verify the product. It answers one question per line: *how will we know we succeeded?* A well-formed specification document contains five distinct element types, each serving a different verification role.

3.1.1 Functional Requirements

A functional requirement states a capability the system must provide, expressed as a verb phrase with a measurable threshold. The threshold is what separates a functional requirement from a vague aspiration.

- **Weak (untestable):** “The system shall sort items quickly.”
- **Strong (testable):** “The system shall classify and divert each item within ≤ 167 ms of entering the sensor gate (throughput ≥ 6 items/min, continuous 1-hour run).”

Every functional requirement must name: (a) the actor or subsystem, (b) the action, (c) the measurable threshold, and (d) the operating condition under which the threshold applies.

3.1.2 Performance Targets

Performance targets quantify quality-of-behaviour: accuracy, latency, bandwidth, efficiency. They differ from functional requirements in that they govern *how well* the function is performed, not merely *whether* it is performed. A sorter that classifies items at 6 items/min but with 40% error satisfies the throughput functional requirement yet fails the accuracy performance target.

3.1.3 Constraints

Constraints are non-negotiable boundaries imposed by physics, regulation, or business. They do not describe desirable behaviour; they eliminate design options outright. Examples: BOM cost $\leq$ \$500 CAD (business constraint), supply voltage from USB $\leq$ 5 V at 2 A (interface constraint), operating temperature 0–50°C (environmental constraint).

3.1.4 Interfaces

Interface specifications describe the boundaries between subsystems, between the product and external systems, and between the product and its users. A mechanical interface names connector types and physical envelope; an electrical interface names voltage levels, connector pinouts, and communication protocols; a software interface names message formats, topics, and timing. Specifying interfaces early prevents integration surprises — the most common source of late-stage schedule overrun.

3.1.5 Acceptance Criteria

Acceptance criteria translate every requirement into a concrete test: apparatus, procedure, pass/fail decision rule. Writing the acceptance test at the same time as the requirement is a discipline check: if you cannot describe how to measure it, the requirement is not yet measurable.

3.1.5.0.1 In Practice. At Vanier’s Product Development Lab, teams have learned to draft acceptance criteria as a three-column table: *Requirement ID*, *Test Procedure* (apparatus + steps), *Pass Condition*. Filling this table during Week 2 surfaces ambiguous requirements before any hardware is ordered, saving the cost of a re-design sprint.

3.2 Requirements vs. Design

The single most persistent error in student (and professional) specification documents is embedding a design decision inside a requirement. Figure 3.1 shows a side-by-side comparison of requirement and design statements for the same capability.

Requirement (What)	Design (How)
Sort accuracy $\geq 95\%$	VL53L1X ToF sensor + threshold classifier
MQTT latency $\leq 200\text{ ms}$	ESP32 with MQTT over TLS
BOM cost $\leq \$500\text{ CAD}$	STM32F4 + off-shelf motor driver

Figure 3.1: Side-by-side comparison of a requirement statement (left) and a design statement (right) for the same capability. Requirements describe observable, measurable outcomes; design statements prescribe implementation choices.

A useful test: replace the specific implementation with an alternative one. If the requirement is still meaningful, it is a genuine requirement. If it becomes nonsensical, it was a design decision dressed up as a requirement.

- **Requirement:** “Classification results shall be transmitted to the cloud dashboard within $\leq 200\text{ ms}$ of each sort event.” Replacing Wi-Fi with Ethernet or cellular still leaves this requirement meaningful. ✓
- **Design-embedded (invalid):** “The MCU shall use MQTT over Wi-Fi to transmit classification results.” This mandates the implementation, closing off alternatives before the team has evaluated trade-offs. ✗

3.2.1 The “What / How” Discipline

Every line of a specification document should survive the question: “Does this describe what the system must do, or does it describe how we plan to do it?” The *what* belongs in the specification. The *how* belongs in the design document, the architecture diagram, or the BOM — produced *after* requirements are frozen.

3.2.1.0.1 In Practice. When reviewing a teammate’s specification draft, highlight every noun that names a specific component (“Arduino”, “ToF sensor”, “Python”) or a specific topology (“mesh network”, “I²C bus”). Each highlighted phrase is a candidate design decision that may be pre-empting a better alternative. Move it to the design rationale section and restate the requirement in terms of observable behaviour.

3.2.1.0.2 Common Pitfalls.

- **Unmeasurable criteria.** “The system shall be user-friendly” cannot be tested. Restate as: “A first-time operator shall complete initial setup in ≤ 10 minutes following the Quick-Start card, with zero support calls, measured on five naïve participants.”
- **Embedding the solution.** “The sensor shall use a VL53L1X time-of-flight module” is a design choice. The requirement should read: “The sensor subsystem shall detect items up to 300 mm from the gate with ≤ 2 mm resolution at throughput ≥ 6 items/min.”
- **Ambiguous conditions.** A requirement with no stated operating condition is incomplete. “Latency ≤ 200 ms” must specify: under what network load? at what ambient temperature? for what message size?

3.3 Risk-Driven Prioritization

Once requirements exist, the team must decide which risks — failures to meet those requirements — deserve the most engineering attention. Without a principled ranking, teams either (a) work on the most visible problem rather than the most dangerous one, or (b) try to mitigate everything simultaneously and mitigate nothing well.

3.3.1 The Basic Risk Equation

The simplest risk model assigns two quantities to each potential failure mode:

- P_{failure} : the estimated probability that the failure occurs in a prototype or field unit, scored 1–10 (1 = very unlikely, 10 = near-certain).
- $C_{\text{consequence}}$: the estimated cost of that failure — in schedule days, dollars, or safety impact — scored 1–10 (1 = trivial, 10 = project-ending).

$$R = P_{\text{failure}} \times C_{\text{consequence}} \quad (3.1)$$

The resulting risk score R ranges from 1 to 100. Teams typically set a threshold (e.g., $R \geq 30$) above which a mitigation plan is mandatory. Figure 3.2 shows a probability–consequence matrix that maps risk scores to action zones.

3.3.2 Limitations of the Two-Factor Model

Equation (3.1) is fast and easy to communicate to non-technical stakeholders, but it conflates two distinct ideas: the *probability* that a failure reaches the user and the *team’s ability to detect* a failure before it causes harm. A failure mode that is severe and somewhat likely but detectable in bench testing is far less dangerous than one that is equally severe but invisible until field deployment. Section 3.4 introduces the FMEA Risk Priority Number, which adds detection as a third factor.

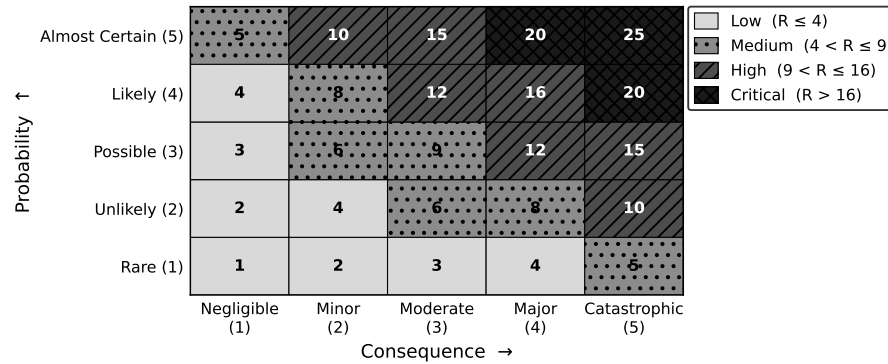


Figure 3.2: Probability × consequence risk matrix for the running example. Cells with $R \geq 30$ fall in the red action zone; $15 \leq R < 30$ in the amber watch zone; $R < 15$ in the green monitor zone.

3.4 FMEA-Style Ranking (RPN)

Failure Mode and Effects Analysis (FMEA) is a structured method for systematically enumerating every way a system can fail, estimating three quantities for each failure mode, and computing a Risk Priority Number to guide mitigation effort.

3.4.1 The RPN Formula

$$\text{RPN} = S \cdot O \cdot D \quad (3.2)$$

where:

- S = Severity (1–10): how bad is the consequence if the failure reaches the user?
- O = Occurrence (1–10): how likely is the failure to occur during normal operation?
- D = Detection (1–10): how difficult is it to detect the failure before it affects the user? *Note: a higher D score means harder to detect.*

RPN ranges from 1 to 1,000. Unlike the two-factor risk score R , the RPN captures the value of test coverage: a team that adds a robust bench test lowers D and therefore lowers RPN even without changing the underlying design.

3.4.1.0.1 In Practice. Industry FMEA tables typically flag $\text{RPN} \geq 100$ as requiring a documented corrective action, and $\text{RPN} \geq 200$ as requiring a design change before release. For a student prototype, a lighter threshold — flag $\text{RPN} \geq 80$ — is appropriate, because prototype runs are short and failures are recoverable.

3.4.2 Scale Anchors

Table 3.1 gives representative scale anchors for S , O , and D as used in this course. Teams should agree on anchors *before* scoring to prevent rater drift.

Table 3.1: Representative 10-point scale anchors for FMEA scoring in a prototype context.

Score	Severity (S)	Occurrence (O)	Detection (D)
1–2	No user impact	Rare (<1 in 1,000 runs)	Immediately obvious; alarms fire
3–4	Minor degradation	Occasional (~1 in 100 runs)	Detectable by inspection or log
5–6	Partial loss of function	Moderate (~1 in 20 runs)	Detectable only with test setup
7–8	Major loss of function	Frequent (~1 in 5 runs)	Difficult; requires special tool
9–10	Safety / total failure	Near-certain (>1 in 2 runs)	Cannot be detected before impact

3.5 The Critical Path of Unknowns

The risk register produced by FMEA answers *which* risks are largest. A separate but related question is *which risks must be resolved first* — because some unknowns block progress on every other subsystem. This is the critical path of unknowns.

Figure 3.3 illustrates the assumption list and dependency arrows for the running example. The highest-RPN item that *also* blocks downstream work defines the prototype’s first experiment.

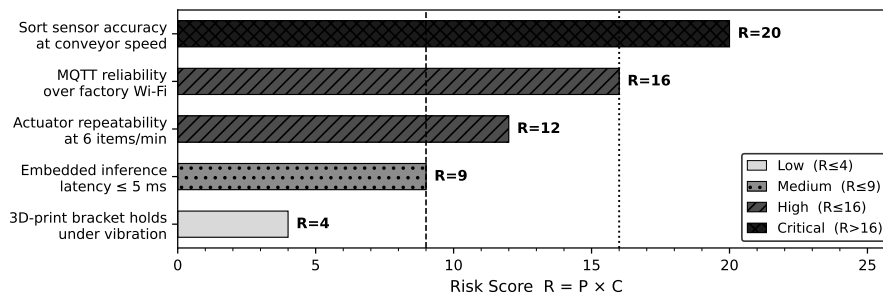


Figure 3.3: Critical unknowns and assumption dependency map for the IoT robotic sorting subsystem. Arrows show which assumptions must be resolved before dependent subsystem work can begin. The sort-sensor assumption (shaded) lies on the critical path because both the divert actuator timing and the firmware logic depend on its output format.

3.5.1 Identifying the Critical Unknown

To find the critical unknown, list every open assumption in the risk register and ask two questions for each:

1. If this assumption turns out to be false, how many other subsystems must change? (Coupling score.)
2. How quickly can we resolve it with a targeted experiment? (Resolvability score.)

An assumption with high coupling and low resolvability is both dangerous and slow to clear — it should head the prototype schedule. An assumption with high coupling but high resolvability should be the *first* experiment run, because a quick test removes the blocker cheaply.

3.5.1.0.1 In Practice. For the IoT sorter, the sort-sensor detection principle (time-of-flight vs. break-beam vs. vision) is the critical unknown: the firmware state machine, the actuator timing, and the MQTT payload format all depend on what the sensor outputs. Running a 48-hour bench test of candidate sensors *before* ordering actuator parts avoids a costly mid-sprint pivot.

3.6 Worked Example: IoT Robotic Sorting Subsystem

This section walks through the full specification and risk-ranking workflow for the running example established in Chapter 3: a connected IoT robotic sorting subsystem classifying and diverting items on a bench conveyor, reporting over MQTT/Wi-Fi, with a target BOM of $\leq \$500$ CAD and a sort accuracy of $\geq 95\%$.

3.6.1 Step 1: Convert Vague Wishes into Measurable Requirements

The team's initial wish list contained five phrases typical of early ideation. Table 3.2 shows each vague phrase converted to a testable requirement.

The most instructive conversion is "fast":

- **Vague:** "The system shall process sensor data fast."
- **Step 1 — name the quantity:** Processing speed is measured in samples per second (kS/s) and as end-to-end latency (ms).
- **Step 2 — anchor to the use case:** At 6 items/min, one item arrives every $60\text{ s}/6 = 10\text{ s}$. A single sort event reads one sensor frame, runs the classifier, and commands the actuator. The conveyor belt speed is 50 mm/s and the item window at the gate is 50 mm, giving a gate dwell time of $50\text{ mm} \div 50\text{ mm/s} = 1,000\text{ ms}$. The actuator must fire within that window.
- **Step 3 — allocate the budget:** Budget allocation: sensor read 1 ms, classifier 2 ms, actuator command 2 ms, margin 995 ms. Conservatively, the end-to-end firmware latency budget is $\leq 5\text{ ms}$. At a 10 kS/s ADC rate, the sensor delivers 10,000 samples per second; one classification frame uses 10 samples, costing $10/10,000 = 1\text{ ms}$ — within budget.
- **Measurable requirement:** "The MCU firmware shall process one classification event (sensor read + classification + actuator command) within $\leq 5\text{ ms}$, measured from sensor-trigger interrupt to actuator-enable GPIO, at a sustained rate of $\geq 10\text{ kS/s}$ sensor sampling."

A sixth implied constraint (power) is added from the hardware context:

CON-01: Total system power draw shall not exceed 5 W when powered from a single USB-C port (5 V, 2 A = 10 W; 5 W provides 50% margin for host compatibility).

Table 3.2: Vague feature wishes converted to measurable, testable requirements for the IoT robotic sorting subsystem. Each requirement identifies actor, action, threshold, and operating condition.

Vague Wish	Measurable Requirement	ID
"Sorts accurately"	The system shall classify items with $\geq 95\%$ accuracy (correct class / total items) over a continuous 100-item run at nominal belt speed.	REQ-01
"Fast"	The MCU firmware shall complete one classification event within ≤ 5 ms (sensor read + classify + actuator command) at ≥ 10 kS/s sensor sampling.	REQ-02
"Enough throughput"	The system shall sustain ≥ 6 items/min throughput for ≥ 60 min without thermal throttling or error accumulation.	REQ-03
"Sends data to the cloud"	Each sort-event MQTT message shall be delivered to the broker within ≤ 200 ms of the sort event, on a 2.4 GHz Wi-Fi network with ≥ -70 dBm RSSI.	REQ-04
"Affordable"	The total prototype BOM (all purchased components) shall not exceed \$500 CAD.	REQ-05

3.6.2 Step 2: Compute RPN for Three Subsystems

The team identifies three high-risk subsystems from Chapter 3's prototype classification: (A) Sort Sensor, (B) MQTT/Wi-Fi Integration, (C) Divert Actuator. For each, the team identifies the primary failure mode, scores S , O , and D on the 1–10 scale from Table 3.1, and computes RPN using Equation (3.2).

Subsystem A — Sort Sensor

- **Failure mode:** Sensor misclassifies item class (false positive or false negative), causing a sort error.
- **Severity (S):** A miss ships a wrong item — significant product defect, customer complaint, possible recall. $S = 8$.
- **Occurrence (O):** Time-of-flight at < 300 mm with a 50 mm/s belt is a moderately proven technique; reflectivity variation across item surface adds uncertainty. $O = 5$.
- **Detection (D):** A classification error is visible in the sort log and confirmed by counting missorted bins at end of run — easily detected during bench test. $D = 2$.
- **RPN:**

$$\text{RPN}_A = S \cdot O \cdot D = 8 \times 5 \times 2 = 80$$

Subsystem B — MQTT/Wi-Fi Integration

- **Failure mode:** MQTT message delivery latency exceeds 200 ms, causing dashboard staleness and potential missed alerts.

- **Severity (S):** Stale dashboard undermines the product's core value proposition (real-time visibility); does not stop physical sorting. $S = 7$.
- **Occurrence (O):** 2.4 GHz Wi-Fi in a lab environment with competing devices experiences frequent retransmissions; MQTT QoS 1 adds handshake overhead. $O = 6$.
- **Detection (D):** Latency can be measured by timestamping publish and subscribe events in the broker log — detectable but requires setup of a monitoring script. $D = 4$.
- **RPN:**

$$\text{RPN}_B = S \cdot O \cdot D = 7 \times 6 \times 4 = \mathbf{168}$$

Subsystem C — Divert Actuator

- **Failure mode:** Actuator fires too late or with insufficient force, failing to divert the item before it exits the gate.
- **Severity (S):** A missed divert is a sort error — same consequence as a sensor misclassification. $S = 8$.
- **Occurrence (O):** Servo or solenoid timing within a 1,000 ms dwell window is well within typical MCU PWM capability; mechanical linkage adds some uncertainty. $O = 3$.
- **Detection (D):** A missed divert is observable by watching the output bins or reading a secondary confirmation sensor — straightforward detection. $D = 2$.
- **RPN:**

$$\text{RPN}_C = S \cdot O \cdot D = 8 \times 3 \times 2 = \mathbf{48}$$

3.6.3 Step 3: Rank and Interpret

Table 3.3 summarises the three RPN scores and maps each to a mitigation action.

Table 3.3: FMEA Risk Priority Number ranking for the three prototype subsystems of the IoT robotic sorting subsystem. RPN computed using Equation (3.2). Subsystems are sorted by descending RPN.

Rank	Subsystem	Failure Mode	S	O	D	RPN	Action
1	MQTT/Wi-Fi	Latency >200 ms	7	6	4	168	Redesign / mitigate
2	Sort Sensor	Misclassification	8	5	2	80	Monitor / mitigate
3	Divert Actuator	Late divert	8	3	2	48	Monitor

Interpretation:

- MQTT/Wi-Fi integration (RPN = 168) is the highest-risk subsystem. The team should prototype and stress-test the Wi-Fi + MQTT stack *first*, before investing time in actuator tuning. Concrete mitigation: run a dedicated latency characterisation script (publish 500 timestamped messages; record broker-side receive timestamp; compute 99th-percentile latency) within Sprint 1.

- Sort Sensor (RPN = 80) exceeds the 80-threshold and requires a documented mitigation plan. The sensor's detection principle should be confirmed in a 100-item accuracy trial before the firmware classifier is written.
- Divert Actuator (RPN = 48) is below the 80-threshold. Standard bench testing during integration is sufficient; no dedicated mitigation sprint is needed.

Applying the basic risk equation (3.1) for cross-check:

$$R_A = P_{\text{failure},A} \times C_{\text{consequence},A} = 5 \times 8 = 40 \quad (\text{action zone})$$

$$R_B = P_{\text{failure},B} \times C_{\text{consequence},B} = 6 \times 7 = 42 \quad (\text{action zone})$$

$$R_C = P_{\text{failure},C} \times C_{\text{consequence},C} = 3 \times 8 = 24 \quad (\text{watch zone})$$

The two-factor scores (3.1) agree with the RPN ranking on the top item (MQTT/Wi-Fi) but obscure the gap between A and B that the detection factor (D) reveals. The sensor is harder to detect as failing, so RPN correctly flags it above the threshold despite its lower occurrence rate.

3.6.4 Try It Yourself

Scenario: A team is developing a **smart irrigation controller** for a rooftop garden. The system reads soil moisture via a capacitive sensor array, decides whether to open a motorised valve, and logs readings to a cloud dashboard over cellular (LTE-M). The initial wish list contains:

- "Waters when needed" — target: soil moisture sensor accuracy $\pm 3\%$ VWC
- "Doesn't over-water" — target: valve response within 30 s of a moisture threshold crossing
- "Cheap to build" — target: BOM $\leq \$300$ CAD

Your tasks:

1. Rewrite each wish as a measurable requirement following the actor–action–threshold–condition format.
2. Identify three failure modes (one per subsystem: moisture sensor, motorised valve, LTE-M uplink).
3. Assign S , O , and D scores on the 1–10 scale with written justification for each score.
4. Compute RPN for each failure mode using Equation (3.2), showing every multiplication step.
5. Rank the three failure modes from highest to lowest RPN and state which subsystem should be prototyped first.
6. Cross-check your top-ranked item using Equation (3.1) and comment on whether the two rankings agree.

Do **not** use the same S , O , D values as the worked example.

Review Questions

1. **(Conceptual)** Explain the difference between a *functional requirement* and a *performance target*, using one example of each from a product domain of your choice. Why must both be present in a complete specification?
2. **(Conceptual)** A colleague argues that specifying the communication protocol (e.g., “shall use Bluetooth Low Energy”) inside a requirement is efficient because it removes ambiguity for the firmware team. Justify why this practice is problematic and describe the correct place in the documentation hierarchy for such a decision.
3. **(Applied — calculation)** A new IoT product team scores three failure modes as follows:
 - Power regulator brown-out: $S = 9, O = 2, D = 3$
 - Firmware deadlock: $S = 6, O = 7, D = 8$
 - Enclosure latch failure: $S = 3, O = 4, D = 1$

Compute RPN for each failure mode using Equation (3.2), showing every multiplication step. Rank them and state which subsystem requires a documented mitigation plan if the threshold is $RPN \geq 80$.

4. **(Applied — multi-step calculation)** A wearable health monitor team sets an acceptance criterion: “Heart-rate measurement error ≤ 3 BPM at 95% confidence over a 10-minute walk test.” They estimate: $P_{\text{failure}} = 4$ (occasional out-of-range readings during vigorous movement), $C_{\text{consequence}} = 8$ (clinical misinterpretation risk). (a) Compute R using Equation (3.1). (b) The team adds a real-time motion-artifact filter that lowers O from 4 to 2. If D remains at 3 and $S = 8$, compute the new RPN and state whether the mitigation moves the failure mode below threshold (Use Equation (3.2)). (c) Explain in one sentence why the two metrics (R and RPN) give complementary information.
5. **(Conceptual)** Define the *critical path of unknowns* and explain how it differs from the highest-RPN item in the risk register. Give an example where the highest-RPN item is *not* on the critical path of unknowns and describe what the team should do first.

End-of-Chapter Artifact

By completing this chapter’s activities, you produce the core of **Milestone 1: the Proposal**. The artifact has three interlocking parts:

1. **Specification Table.** A table of 3–5 measurable requirements (functional requirements + performance targets) and any binding constraints, formatted with columns: ID, Requirement Statement, Threshold, Operating Condition, Acceptance Test. Each row must be testable as written.
2. **Risk Register.** An FMEA table listing at least three failure modes (one per high-risk subsystem identified in Chapter 3). Each row contains: Subsystem, Failure

Mode, S , O , D , RPN, Rank, and Mitigation Plan. The table is sorted by descending RPN. Failure modes above RPN 80 must have a specific mitigation action tied to a sprint.

3. **Scoping Rationale.** One paragraph (100–150 words) that names: (a) the central prototype question, (b) the highest-risk assumption (the critical unknown), and (c) the first experiment that will resolve that assumption. This paragraph becomes the executive summary of your Milestone 1 submission.

Together, these three parts answer the investor’s foundational question: “*What will this thing do, how will you know it works, and what could kill the project?*”

Summary

This chapter introduced the anatomy of a specification — functional requirements, performance targets, constraints, interfaces, and acceptance criteria — and showed how each element must be measurable and testable. It established the discipline of separating *what* (requirements) from *how* (design), a distinction that keeps early design choices from foreclosing better alternatives. Two complementary risk tools were presented: the two-factor risk equation $R = P_{\text{failure}} \times C_{\text{consequence}}$ (Equation (3.1)) for rapid stakeholder communication, and the FMEA Risk Priority Number $RPN = S \cdot O \cdot D$ (Equation (3.2)) for engineering-level prioritisation that accounts for detection difficulty. Applied to the IoT robotic sorting subsystem, the analysis ranked MQTT/Wi-Fi integration ($RPN = 168$) as the first prototype target, followed by the sort sensor ($RPN = 80$), and confirmed that the divert actuator ($RPN = 48$) can proceed with standard integration testing.

You now have a specification table, a risk register, and a scoping rationale — the three components of the Milestone 1 Proposal. Chapter 5 takes this artifact as its starting point and shows how to translate the highest-risk assumptions into a *prototype plan*: selecting fidelity levels, designing the first experiment, and scheduling the sprint sequence that will resolve the critical unknowns in the right order.



INDEX

- acceptance criterion, 28
- B2B, 7
- B2C, 7
- build-to-order, 7
- business model, 6
- commodity high-volume, 7
- consequence, 30
- constraint, 28
- cost, 6
- critical path of unknowns, 32
- detection, 31
- documentation layer, 22
- effort
 - relative, 20
- electrical interface, 28
- engineer, 4
- extrinsic value, 14
- Failure Mode and Effects Analysis, 31
- fidelity, 20
- firm size, 5
- FMEA, 31
- functional requirement, 27
- go/no-go, 18
- hardware-as-a-service, 7
- integration, 17
- interface specification, 28
- intrinsic value, 14
- iteration
 - controlled, 18
 - uncontrolled, 18
- lab track, 2
- large enterprise, 5
- licensing, 7
- mechanical interface, 28
- milestone, 3
- occurrence, 31
- performance target, 28
- pitfall
 - over-fidelity, 22
 - prototype as mini-product, 22
- probability of failure, 30
- product, 3
- product sale, 6
- production volume, 6
- project context brief, 10
- proof-of-concept, 1
- prototype, 3
 - definition, 14
 - functional, 19
 - looks-like/works-like, 19
 - pre-production, 19
 - proof-of-concept, 19
- Prototype Development Cycle, 13
- prototype plan, 16
- requirements vs. design, 29
- revenue model, 6
- risk
 - in prototyping, 14
 - list, 15
- risk-driven prioritization, 30
- running example, 8
- seven-stage workflow, 2
- severity, 31
- SME, 5
- software interface, 28
- specialty low-volume, 7
- specification, 15, 27
- start-up, 5
- subsystem, 16
- test for requirements, 29
- theory track, 2

time, 6

unit economics, 18

validation, 17

what/how discipline, 29